
기초 수학과 통계

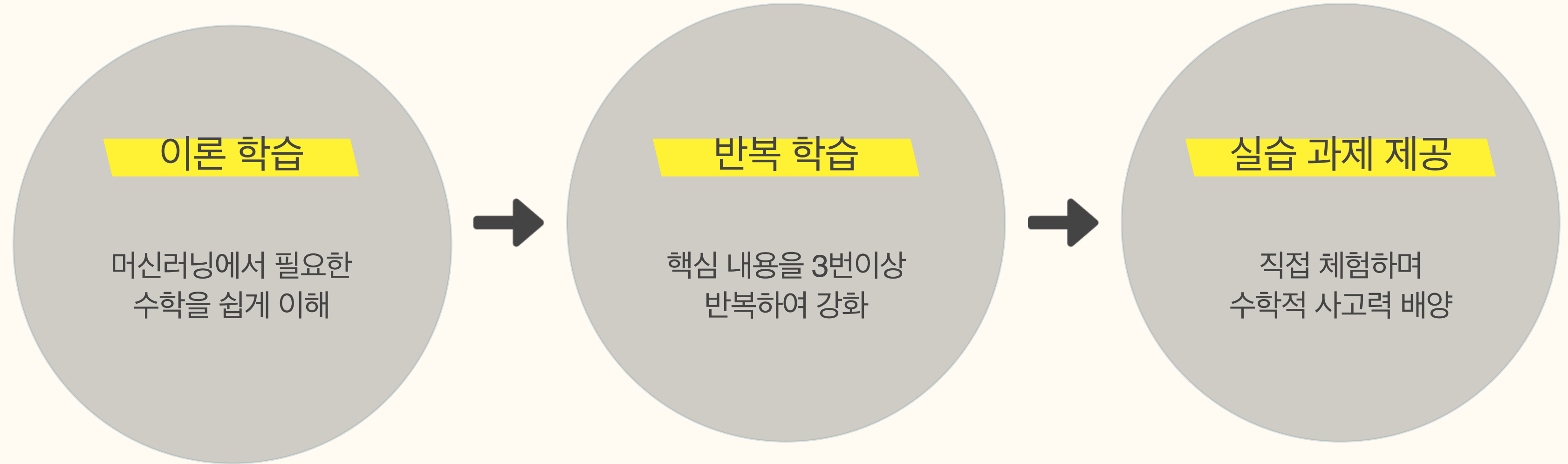
수포자도 이해하는 AI 수학

강의 목적

머신러닝에 필요한 수학적 사고를 배우고,
AI 모델이 어떻게 의료 예측을 수행하는지 논리적으로 이해
할 수 있는 능력을 기른다.

목차

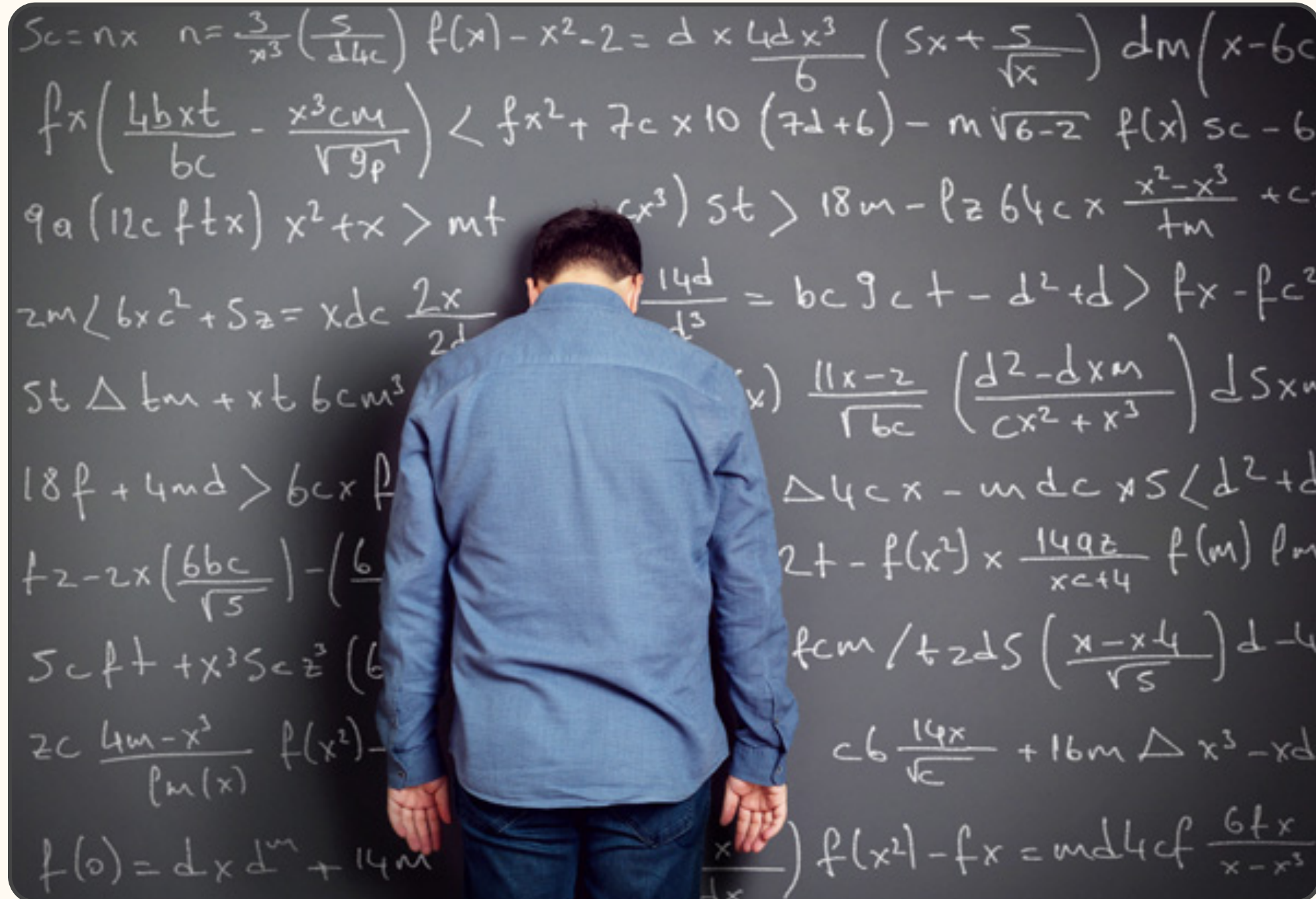
1. 수학을 배우는 이유
AI에 필요한 수학적 사고의 중요성과 의료 데이터 생성 원리
2. 벡터와 행렬
강의에서 시각적 자료를 활용하여 학습자의 이해도를 높이는 방법
3. 가중합과 선형방정식
AI 예측의 기본 원리인 가중합 계산과 선형방정식 이해
4. AI예측 핵심 함수 4가지
선형, 지수, 로그, 시그모이드 함수의 활용과 예측 원리



이론→적용→실습 구조로 체계적으로 학습하세요.

수학을 배우는 이유

머신러닝은 숫자놀이이다



수학

무엇이 떠오르시나요?

복잡한 공식들, 어려운 계산, 막막한 숫자들...
하지만 정말 그럴까요?



일상에서의 수학

사실 우리는 이미 수학을 하고 있습니다

친구들과 더치페이 할 때 1인당 얼마씩 낼지 계산하고, 마트에서 할인상품과 2+1 행사제품 중 어느 게 더 저렴한지 비교합니다.

이것이 바로 실생활 속 수학입니다.

어느것이 더 저렴할까요?

과자를 사려고 합니다,

A마트: 3,000원 (정가)

B마트: 4,000원 (30% 할인)



수학적으로 표현하면

A마트: 3,000원 (정가)

B마트: 4,000원 $\times$ 0.7 = 2,800원

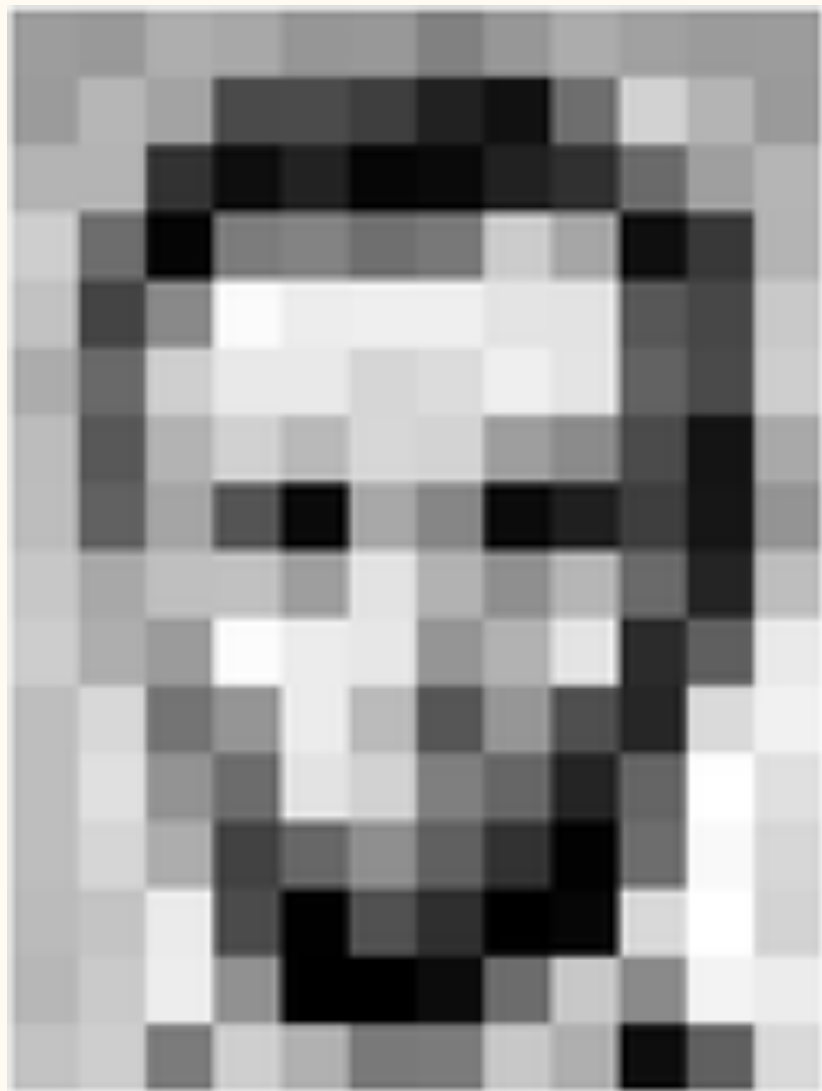
→ B마트가 200원 더 저렴!





AI에서 수학을 왜 배워야 할까?

AI는 어떻게 이미지를 학습할까?



AI는 어떻게 이미지를 학습할까?



머신러닝 모델



색의 밝기를 기준으로
숫자로 변환

187	183	176	168	160	152	129	181	173	143	166	166
196	182	163	74	75	42	88	107	118	210	180	164
188	180	60	14	14	4	10	93	48	104	166	181
206	108	5	104	131	111	120	204	164	15	96	180
194	68	187	281	237	236	206	228	227	67	71	201
173	106	207	203	233	214	220	206	228	96	76	206
184	68	176	209	185	216	211	158	139	75	20	169
189	97	166	84	10	166	156	111	31	62	22	148
189	166	161	163	168	227	179	143	162	106	36	180
206	176	166	262	236	231	169	176	228	63	95	204
196	214	156	149	236	167	85	159	75	36	216	241
196	214	147	168	227	210	127	103	36	161	255	204
196	214	173	66	133	142	96	96	2	106	249	218
187	196	206	76	1	61	47	6	4	217	266	211
183	202	237	149	9	9	12	106	299	136	243	206
196	206	129	207	177	121	129	206	175	13	96	218

AI는 어떻게 이미지를 학습할까?



머신러닝 모델



컴퓨터는 숫자만 이해할 수
있기때문

187	183	176	168	160	152	129	187	172	147	188	186
188	182	143	74	75	42	30	17	118	218	180	164
180	180	50	14	34	4	10	33	48	104	188	181
204	109	6	124	131	111	120	204	186	18	94	180
194	48	107	281	237	236	236	228	227	87	71	201
172	106	207	233	233	214	220	208	238	98	78	206
188	88	179	208	185	214	211	188	138	75	30	168
189	97	145	84	10	148	134	11	31	42	22	148
188	148	191	193	198	227	178	143	182	104	36	190
204	178	186	282	236	231	148	178	238	43	95	208
190	214	114	148	236	187	84	180	78	38	218	247
190	234	147	108	237	214	127	102	34	191	254	224
190	214	179	86	103	143	94	80	2	108	248	215
187	194	206	78	1	81	47	0	6	217	288	211
183	202	207	145	0	0	12	108	208	138	243	206
194	204	129	207	177	121	129	208	178	13	94	218

AI는 어떻게 이미지를 학습할까?



머신러닝 모델



컴퓨터는 숫자만 이해할 수 있기때문

[illegible]



야구는
투수놀음

머신러닝은 숫자놀이



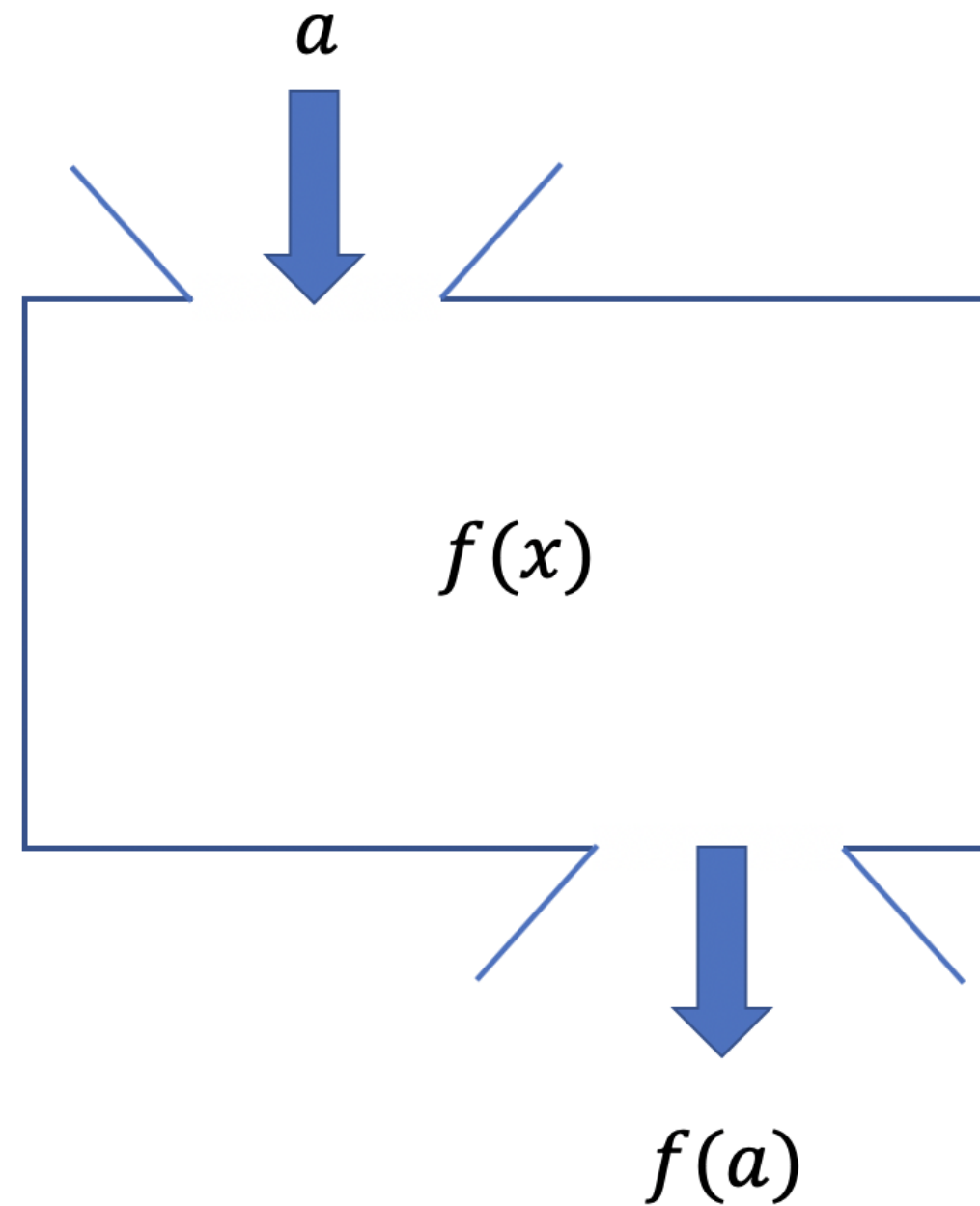


“수포자”인 내가 할수있을까?



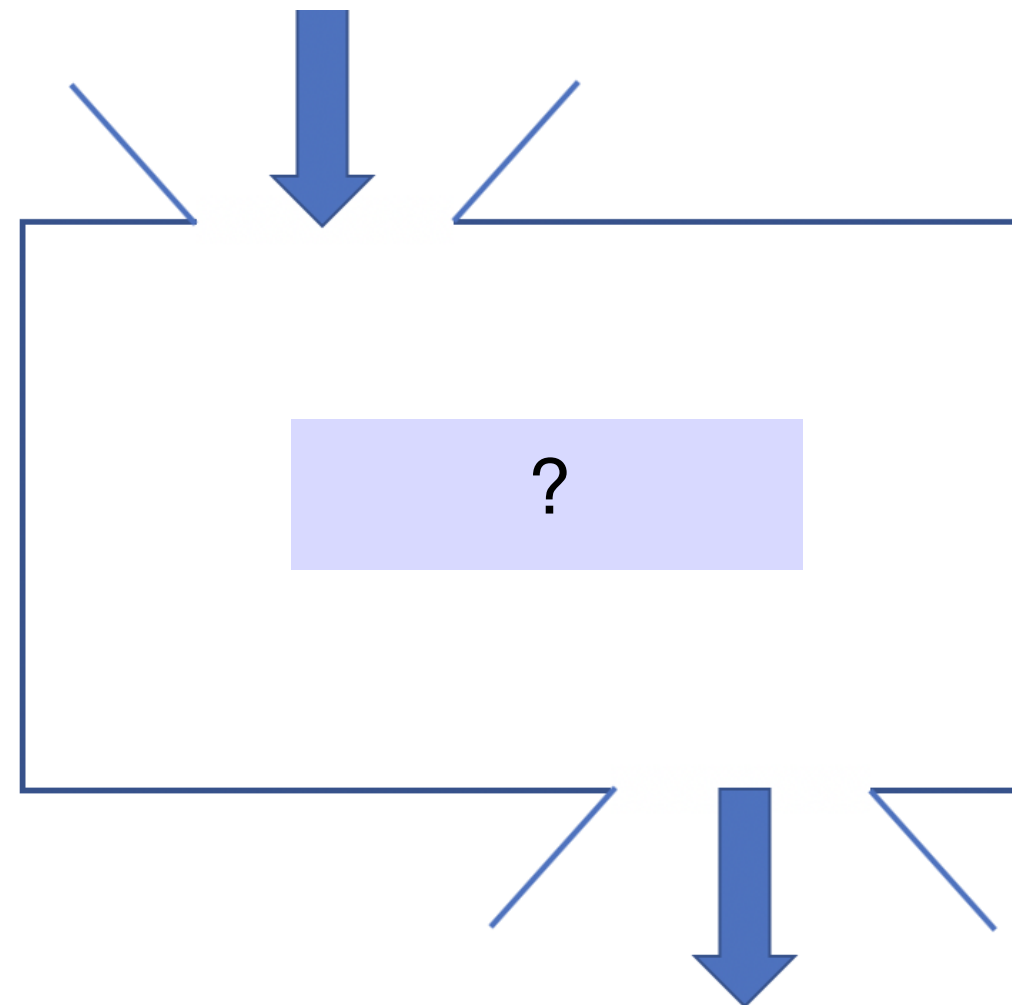
머신러닝은 무엇을 **학습**할까?

AI 모델도 결국 특정 입력(특성)을 받아 특정 출력(예측)을 만드는 함수



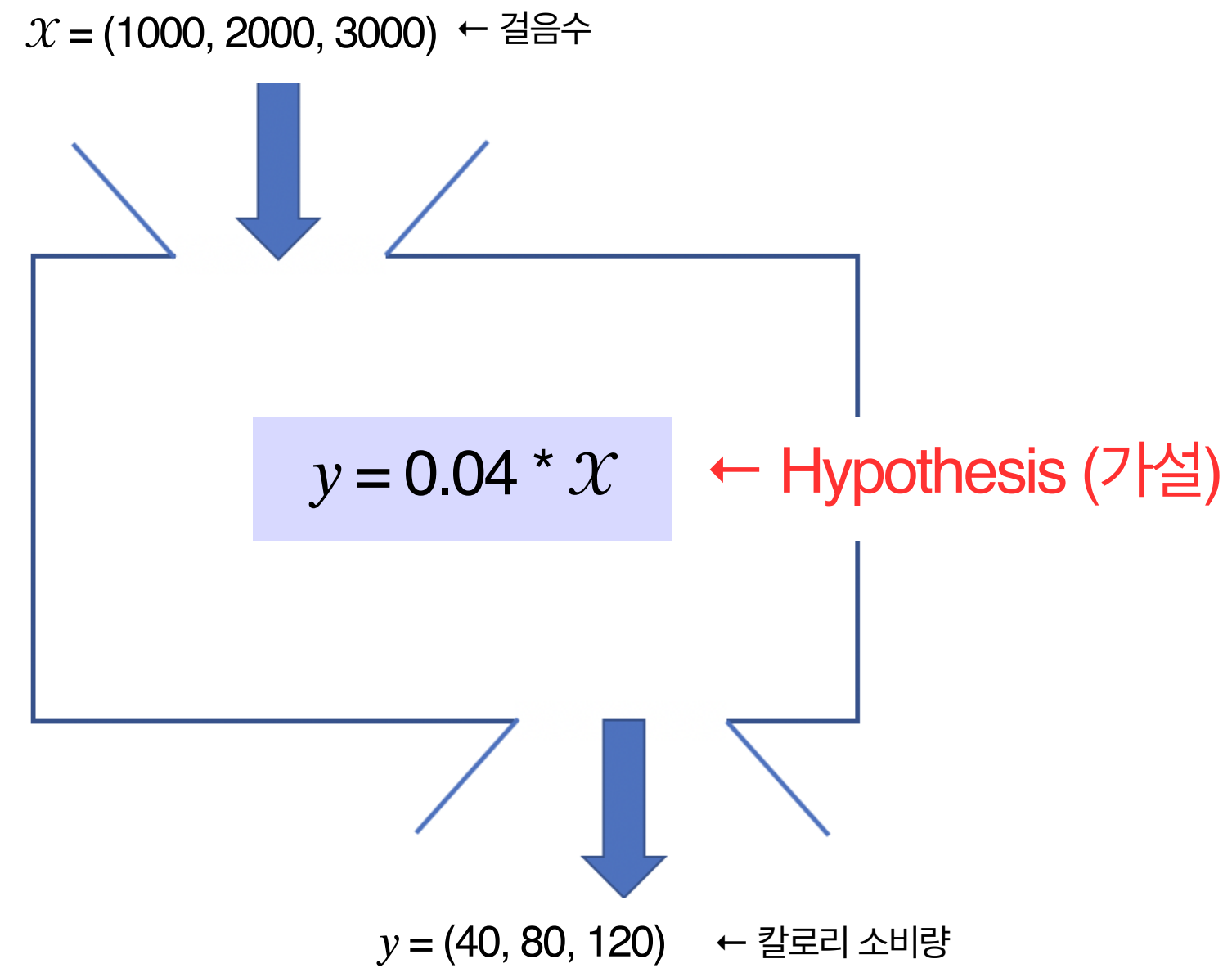
박스에는 어떤 규칙이 들어있을까?

$x = (1000, 2000, 3000)$ ← 걸음수



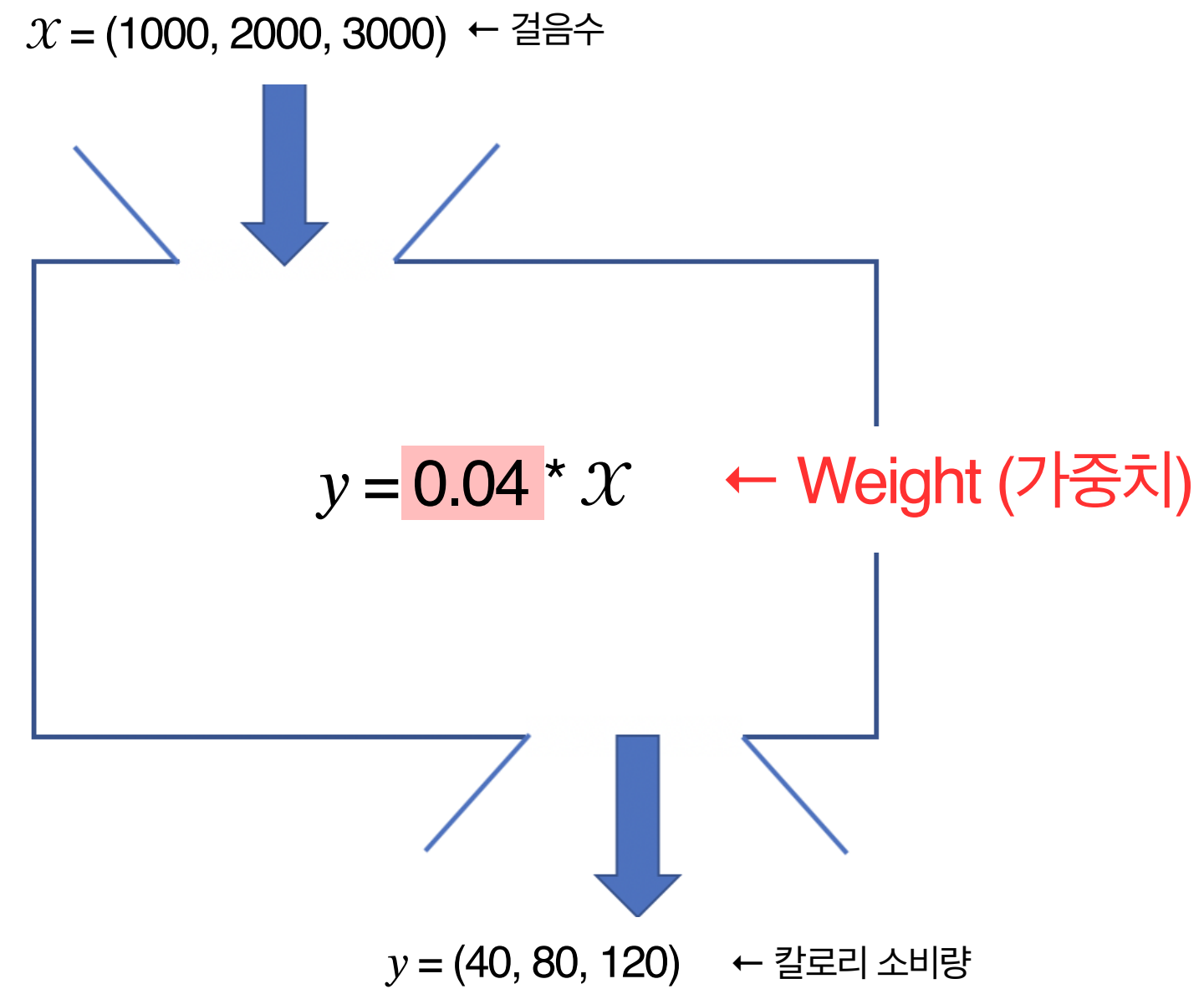
$y = (40, 80, 120)$ ← 칼로리 소비량

머신러닝은 데이터간의 상관관계를 학습



* 최적의 weight를 찾는 과정 *

머신러닝은 데이터간의 상관관계를 학습





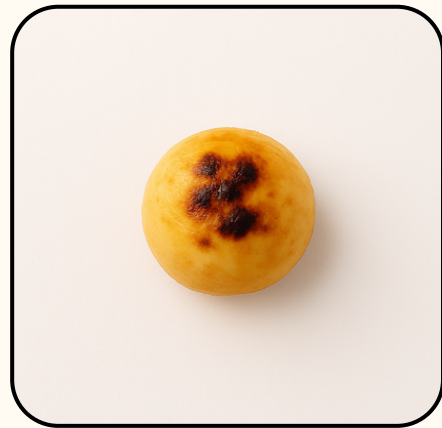
*min Cost **Weight***

머신러닝은 cost를 최소화하는 **weight**를 학습한다
오차

벡터와 행렬

꼬치와 도시락

스칼라 scalar



하나의 숫자 값
크기만 존재하여 점으로 표현 안됨

1

벡터 vector



크기와 방향을 가지는 수학적 양.
하나의 데이터, 점

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

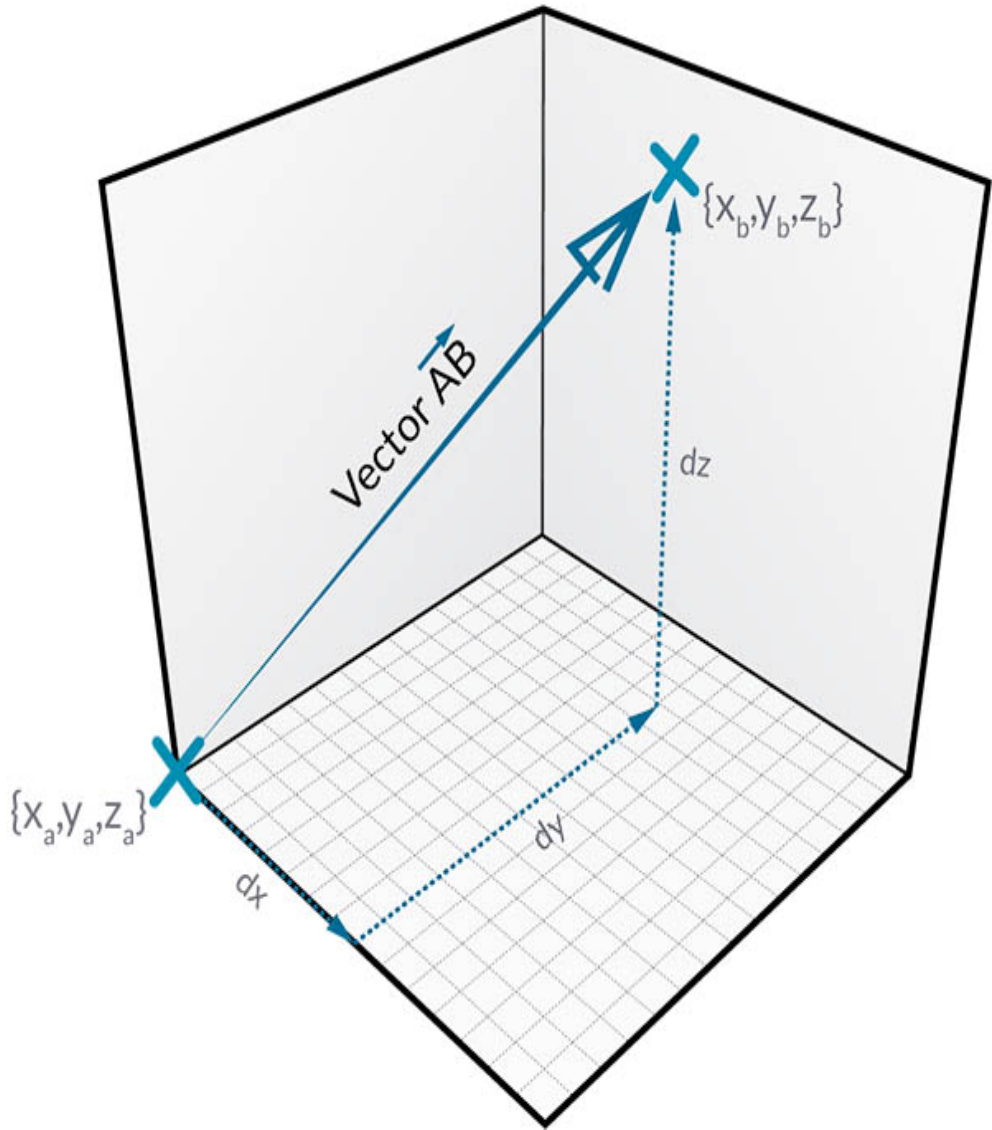
행렬 matrix



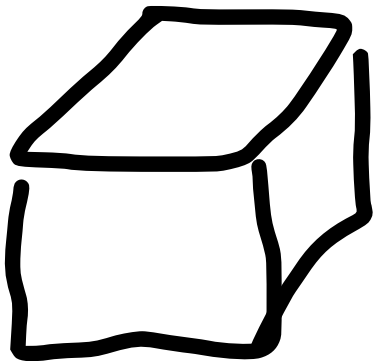
벡터를 여러개 묶은 것.
복수의 데이터, 점의 집합

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

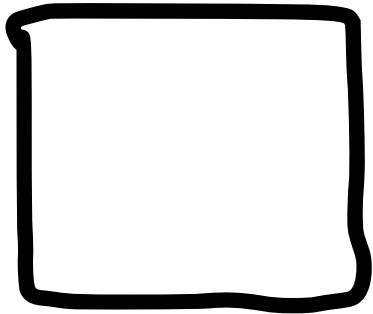
벡터



Vector $\overrightarrow{AB} = \{d_x, d_y, d_z\} = \{x_b - x_a, y_b - y_a, z_b - z_a\}$



Tensor



matrix

스칼라

환자의 나이: 45, 환자의 현재 체온: 37.5

벡터

환자 정보: [키, 체중, 나이]

행렬

여러 환자들의 정보 모음 [키1, 체중1, 나이1], [키2, 체중2, 나이2], ...



코드로 작성해보기



파이썬 리스트 vs Numpy

파이썬 리스트

파이썬의 기본 데이터 구조로 순서가 있는 요소들의 집합

- 데이터타입: Python List
- Dynamic typing: 다양한 타입의 데이터를 자유롭게 저장
- 메모리 오버헤드: 메모리에 흩어져서 저장되기 때문에 포인터 필요
- 연산 속도: 각 요소를 하나씩 처리해야 하기 때문에 느림

```
# 환자의 이름, 나이, 체중, 입원여부를 한 번에 저장 가능
```

```
patient_info = ['김영수', 45, 70.5, True]
```

↓ ↓ ↓ ↓

문자열 정수 실수 참거짓

(string) (int) (float) (boolean)

Numpy 배열

파이썬의 고성능 수치 계산을 위한 핵심 라이브러리

- 데이터타입: NumPy Array
- Static typing: 동일한 타입의 데이터만 저장 (주로 숫자)
- 메모리 효율성: 연속된 메모리 블록에 저장되어 효율적
- 연산 속도: 벡터화 연산으로 전체 배열을 한 번에 처리해서 빠름

```
import numpy as np
# 환자의 나이, 체중, 키, 혈압을 숫자로만 저장
patient_data = np.array([45, 70.5, 175, 120])
                        ↓   ↓   ↓   ↓
                        나이 체중 키  혈압
```

머신러닝에서 Numpy가 필수인 이유

AI 모델은 수백만 개의 숫자 데이터를 동시에 처리해야 합니다.
머신러닝에서는 속도가 생명이기 때문에 NumPy 없이는 불가능합니다!

파이썬 리스트 vs Numpy 배열

구분	파이썬 리스트	NumPy 배열
정의	파이썬 기본 데이터 구조	고성능 수치 계산 라이브러리
데이터 타입	다양한 타입 혼재 가능	동일한 타입만 저장
메모리 사용	흩어져서 저장 (비효율적)	연속 블록 저장 (효율적)
연산 속도	느림 (하나씩 처리)	빠름 (벡터화 연산)
의료 활용	환자 기본 정보 목록	대용량 생체 신호 분석
예시	<code>['김철수', 45, True]</code>	<code>[45, 70.5, 175, 120]</code>



Numpy 작성해보기

내적과 행렬곱

내적 (Dot product)이란?

두 벡터의 대응 원소끼리 곱해서 모두 더한 값

- 원소별로 곱한 뒤 모두 더한다
- 결과는 숫자 하나 (스칼라)

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 32$$

내적 결과값: $(1 \times 4) + (2 \times 5) + (3 \times 6) = 32 \rightarrow 1 \times 1$

행렬곱 (Matrix Multiplication)이란?

두 행렬의 내적을 한 번에 계산하는 방법

- 첫 번째 행렬의 각 행과 두 번째 행렬의 각 열을 내적한 결과들의 모음
- 결과는 각 위치에 대응하는 행과 열의 내적값 (행렬)

2	1	0
1	3	1
0	2	4

 $\times$

3	1	-1
2	0	1
1	4	2

 $=$

A		

$$A = (2 \times 3) + (1 \times 2) + (0 \times 1) = 6 + 2 + 0 = 8$$

행렬곱 (Matrix Multiplication)이란?

2	1	0
1	3	1
0	2	4

 $\times$

3	1	-1
2	0	1
1	4	2

 $=$

8	2	-1
10	5	4
8	16	10

$$A_{11} = (2 \times 3) + (1 \times 2) + (0 \times 1) = 6 + 2 + 0 = 8$$

$$A_{12} = (2 \times 1) + (1 \times 0) + (0 \times 4) = 2 + 0 + 0 = 2$$

$$A_{13} = (2 \times -1) + (1 \times 1) + (0 \times 2) = -2 + 1 + 0 = -1$$

$$A_{21} = (1 \times 3) + (3 \times 2) + (1 \times 1) = 3 + 6 + 1 = 10$$

$$A_{22} = (1 \times 1) + (3 \times 0) + (1 \times 4) = 1 + 0 + 4 = 5$$

...

이런 식으로 각 행 $\times$ 열을 내적하면 됩니다.

행렬곱의 핵심 규칙

★ 첫 번째 행렬의 열 개수 == 두 번째 행렬의 행 개수

4	1
0	3
2	5

 $\times$

2	7
1	4
3	6

(3 × 2 행렬) × (3 × 2 행렬) → 불가능

4	1
0	3
2	5

 $\times$

1	2	3
2	1	0

(3 × 2 행렬) × (2 × 3 행렬) → 가능

행렬곱의 핵심 규칙

★ 행렬곱의 결과는 **A의 행의 갯수 x B의 열의 갯수**

4	1
0	3
2	5

A

×

1	2	3
2	1	0

B

=

6	9	12
6	3	0
12	9	6

(3 × 2 행렬) × (2 × 3 행렬) → (3 × 3)

환자 데이터: [45, 85, 25] # 나이(세), 체중(kg), BMI
학습된 가중치: [0.02, 0.015, 0.08]

내적 계산:

$$\begin{aligned} &45 \times 0.02 + 85 \times 0.015 + 25 \times 0.08 \\ &= 0.9 + 1.275 + 2.0 \\ &= 4.175 \text{ (위험도 점수)} \end{aligned}$$

내적과 AI 예측

환자 정보 $\times$ 가중치(weight) = AI 예측값

AI 모델의 핵심 과정

1. 환자 정보를 숫자 벡터로 변환
2. 학습된 가중치와 내적 계산
3. 예측값 도출

내적: 각 요소의 중요도를 반영한 종합 점수

건강검진 문항표

아래 질문들은 “생활 패턴이 칼로리 소비에 얼마나 영향을 주는지”를 평가하기 위한 건강검진형 설문입니다.

문항	질문 내용	당신의 점수 (0~5)	AI 가중치 (중요도)	점수 × 가중치
Q1	하루 동안 얼마나 많이 걷거나 움직이나요?	4	1.30	5.20
Q2	스트레스를 받으면 몸이 쉽게 피로해지나요?	3	0.90	2.70
Q3	수면의 질은 어떤가요? (좋을수록 점수 높음)	2	0.40	0.80
Q4	평소 집중 시간이 긴 편인가요?	1	0.20	0.20
Q5	파란색을 좋아하나요?	5	0.05	0.25
가중합(내적 결과)				9.15

AI 예측 하루 소비칼로리
9.15 + 1200 ≈ 1209 kcal

$x = [4, 3, 2, 1, 5]$

·

$w = [1.30, 0.90, 0.40, 0.20, 0.05]$

=

$\hat{y} = 9.15$

환자 데이터 (3명):

[[65, 75, 160], # 환자1: 나이, 체중, 키
[45, 68, 165], # 환자2: 나이, 체중, 키
[55, 82, 170]] # 환자3: 나이, 체중, 키

학습된 가중치: [1.2, 0.8, 0.3]

행렬곱 계산:

환자1 예측: $65 \times 1.2 + 75 \times 0.8 + 160 \times 0.3 = 186$

환자2 예측: $45 \times 1.2 + 68 \times 0.8 + 165 \times 0.3 = 158.9$

환자3 예측: $55 \times 1.2 + 82 \times 0.8 + 170 \times 0.3 = 182.6$

결과값: [186, 158.9, 182.6] # 각 환자의 예측 혈압

여러 환자 데이터 → 행렬곱

여러 환자를 처리하는 AI의 핵심 연산

- 환자 한명: 벡터(한명 데이터) $\times$ 벡터(가중치) = 내적
- 환자 여러 명: 행렬(여러환자 데이터) $\times$ 벡터(가중치) = 행렬곱

```
# 환자 데이터 행렬 생성 (3명의 환자, 각각 나이/체중/키)
patient_data = np.array([
    [65, 75, 160], # 환자1: 나이, 체중, 키
    [45, 68, 165], # 환자2: 나이, 체중, 키
    [55, 82, 170]  # 환자3: 나이, 체중, 키
])

# 학습된 가중치 벡터
weights = np.array([1.2, 0.8, 0.3])

# 행렬곱으로 모든 환자의 예측값을 한 번에 계산
predictions = np.dot(patient_data, weights)
```

np.dot()

Numpy의 내적 또는 행렬 곱셈을 수행하는 함수

1차원 벡터는 내적,
2차원 벡터는 행렬 곱셈을 수행

환자 데이터: [45, 85, 25] # 나이(세), 체중(kg), BMI
학습된 가중치: [0.02, 0.015, 0.08]

내적 계산:

$$\begin{aligned} &45 \times 0.02 + 85 \times 0.015 + 25 \times 0.08 \\ &= 0.9 + 1.275 + 2.0 \\ &= 4.175 \text{ (위험도 점수)} \end{aligned}$$

내적과 AI 예측

환자 정보 $\times$ 가중치 = AI 예측값

AI 모델의 핵심 과정

1. 환자 정보를 숫자 벡터로 변환
2. 학습된 가중치와 내적 계산
3. 예측값 도출

내적: 각 요소의 중요도를 반영한 종합 점수

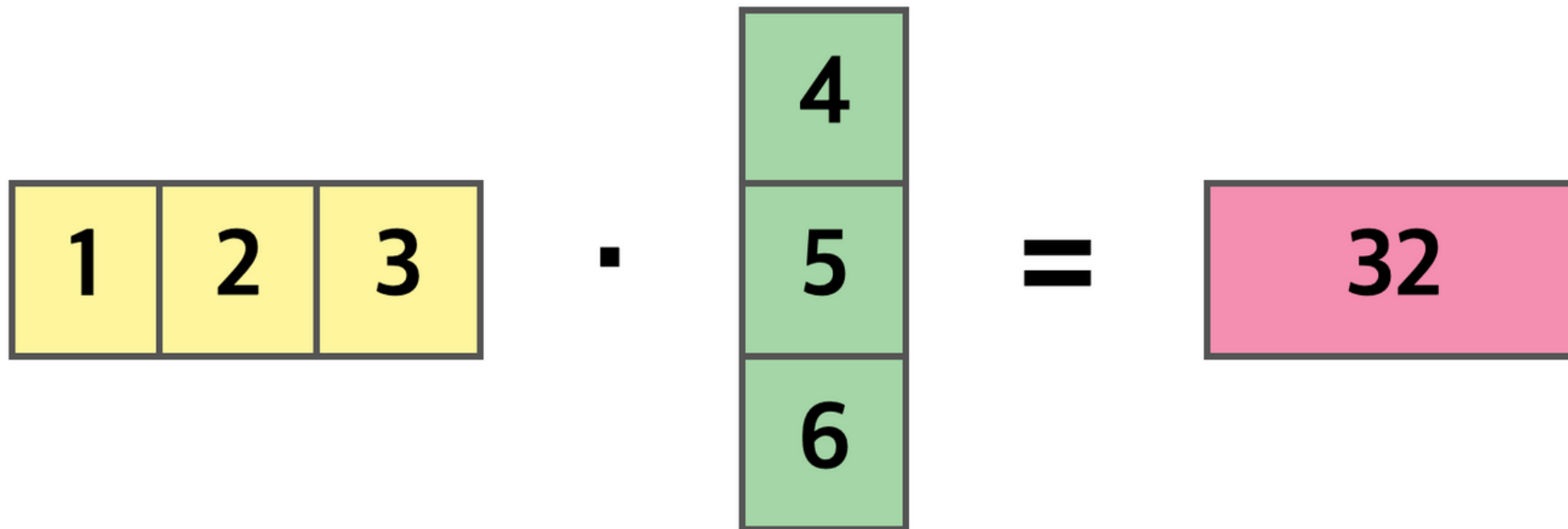
가중합과 선형방정식

AI 예측의 핵심원리

AI 예측의 핵심: 가중합

각 데이터에 **가중치(중요도)**를 곱해서 모두 더한 값

AI가 예측을 할 때 가장 먼저 사용하는 계산 방식이 바로 가중합(Weighted Sum)입니다.
즉, 환자 정보 × 가중치의 형태로 이루어져 있습니다.


$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 32$$

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n$$

가중합을 구현하는 수학적 방식이 내적

선형방정식(Linear Equation)이란?

가중합에 **편향(bias)**을 더한 완전한 예측 모델

편향은 모든 입력값이 0이어도 가져야 할 기본값입니다.
예를들어, 운동량이 0이어도 기초대사율로 기본 1200kcal는 소모됩니다.
이 값이 편향입니다.

가중합

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n$$

기본 계산

+

편향

$$+ b$$

기본값 보정

=

선형방정식

$$y = wx + b$$

완전한 모델

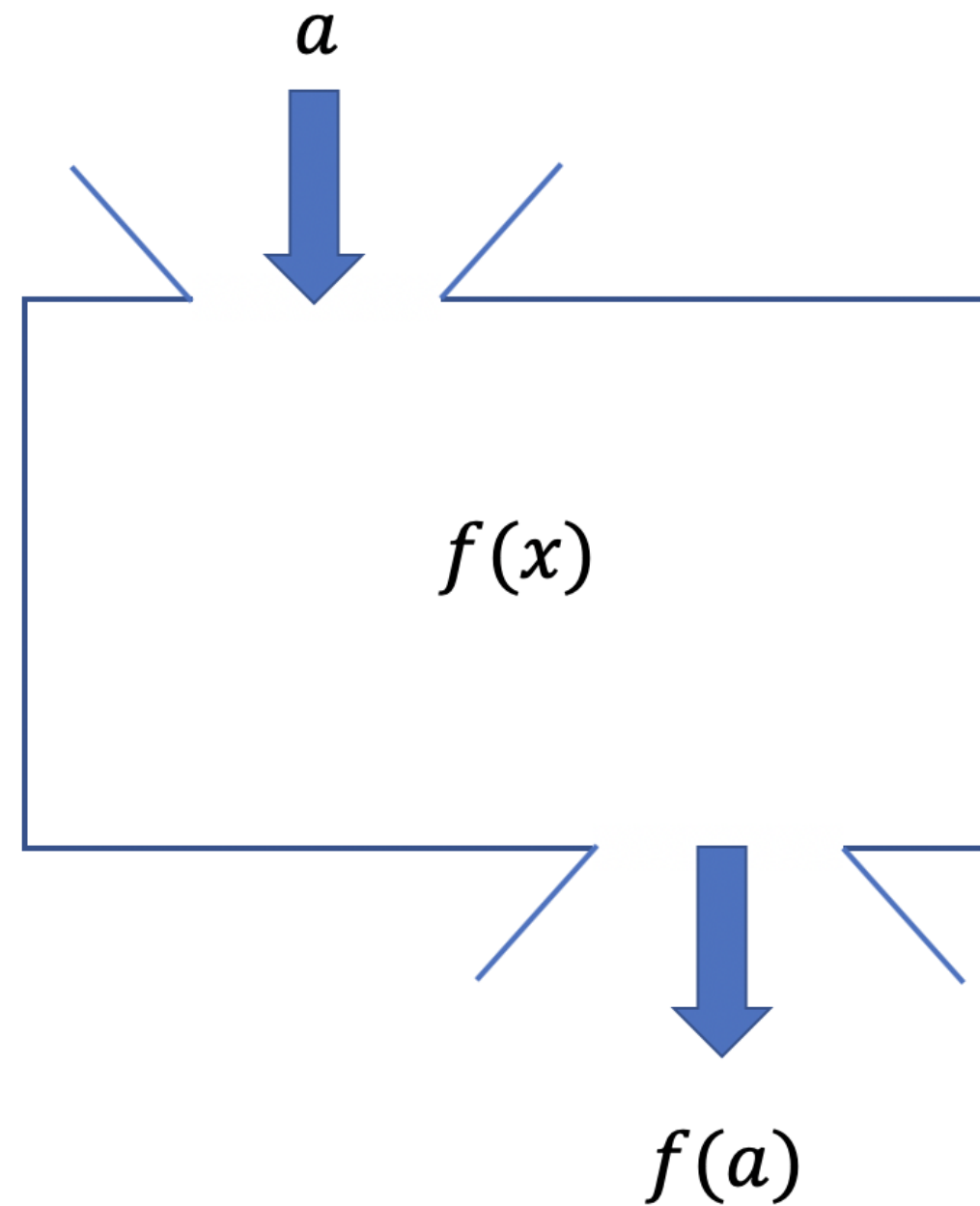


코드로 작성해보기

머신러닝 핵심 함수 4가지

AI 동작원리를 이해하는 필수 개념

함수란 입력값을 넣으면 정해진 규칙에 따라 출력값을 만들어주는 기계

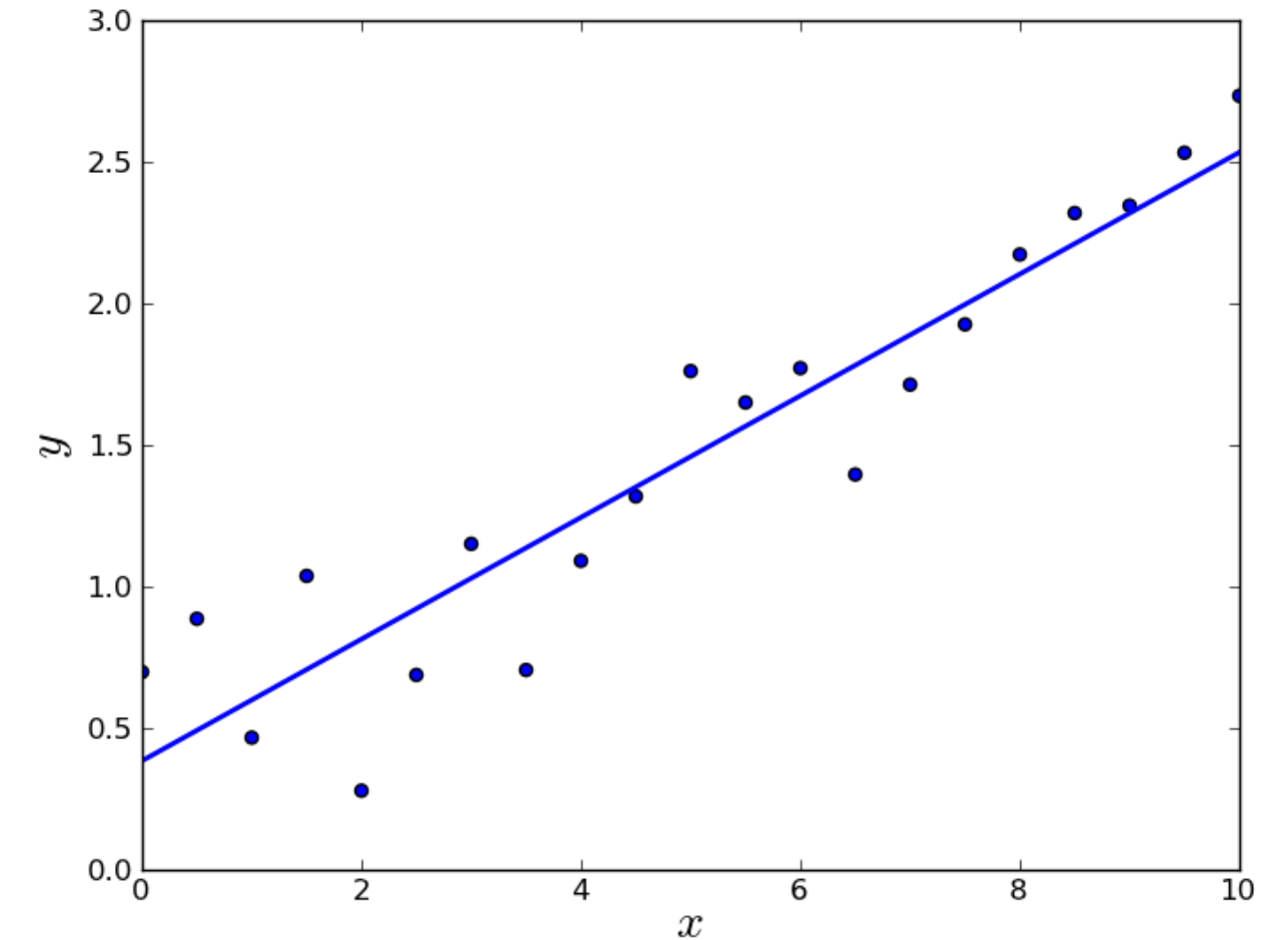


선형함수 (Linear Function)

공식: $y = ax + b$

가장 기초적이며 AI 모델의 기본이 되는 함수입니다.
직선형태로 입력이 증가하면 출력도 일정 비율로 증가/감소합니다.

- 대부분의 예측 모델의 기본구조로 사용되는 뼈대가 되는 함수
- 운동 시간이 늘어날수록 스트레스 지수가 줄어드는 관계
- 시급 계산: 시간당 급여 $\times$ 근무시간



어디에 활용될까?

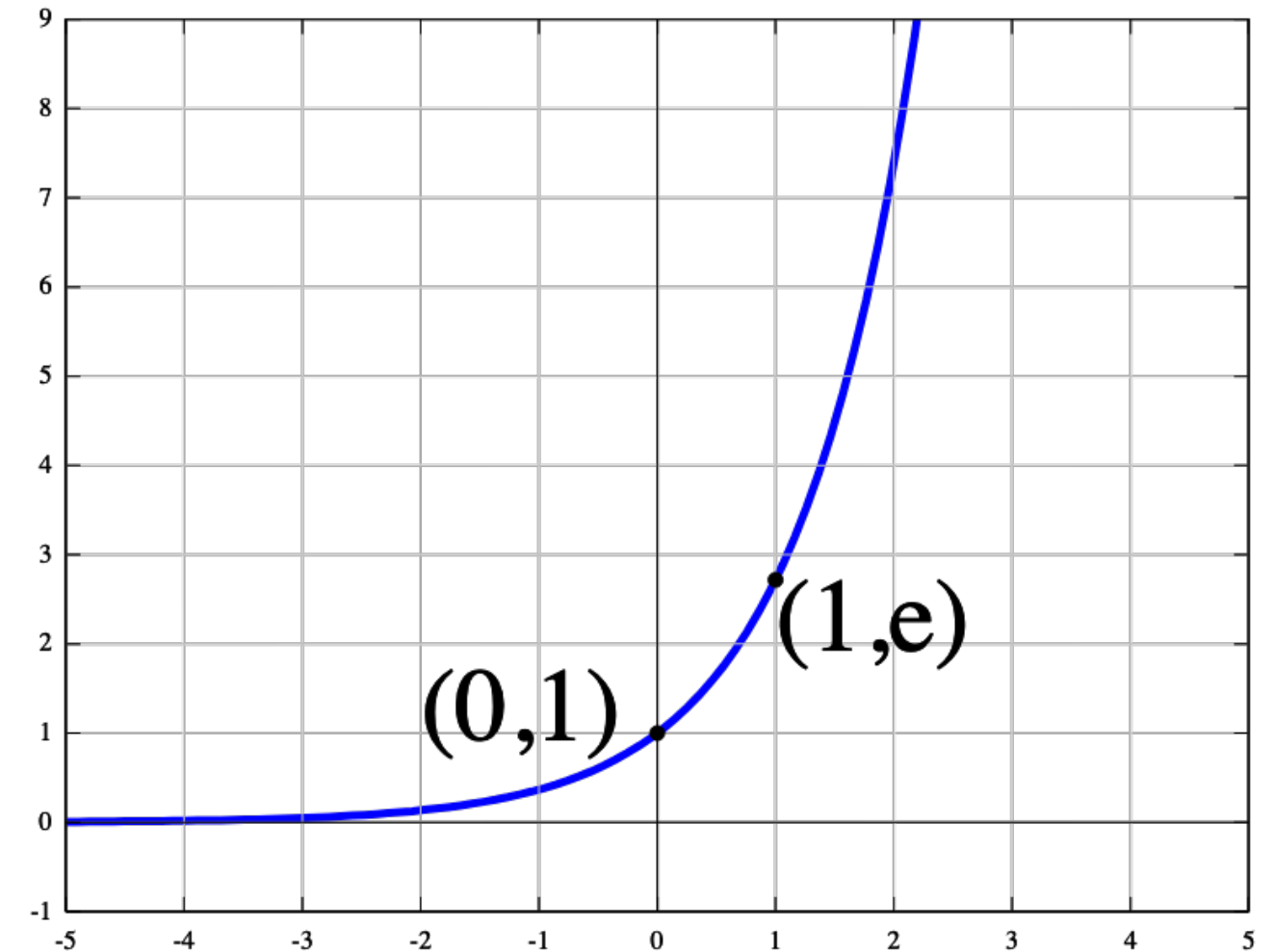
- 혈압 예측: 나이, BMI, 흡연 여부로 미래 혈압 수치 예측
- 혈당 변화 예측: 식습관·활동량·체중으로 다음 검사 시 혈당값 예측

지수함수 (Exponential Function)

공식: $y = e^x$

급격한 성장 패턴을 표현하는 가장 강력한 함수입니다.
처음엔 천천히 나중엔 폭발적으로 증가/감소하는 패턴을 표현합니다.

- 바이러스나 세포처럼 급속도로 증가하는 현상을 모델링
- 감염자가 1명에서 2명, 4명, 8명으로 배수 증가하는 관계
- 세금 계산: 소득 구간별로 누진세율이 기하급수적 증가



어디에 활용될까?

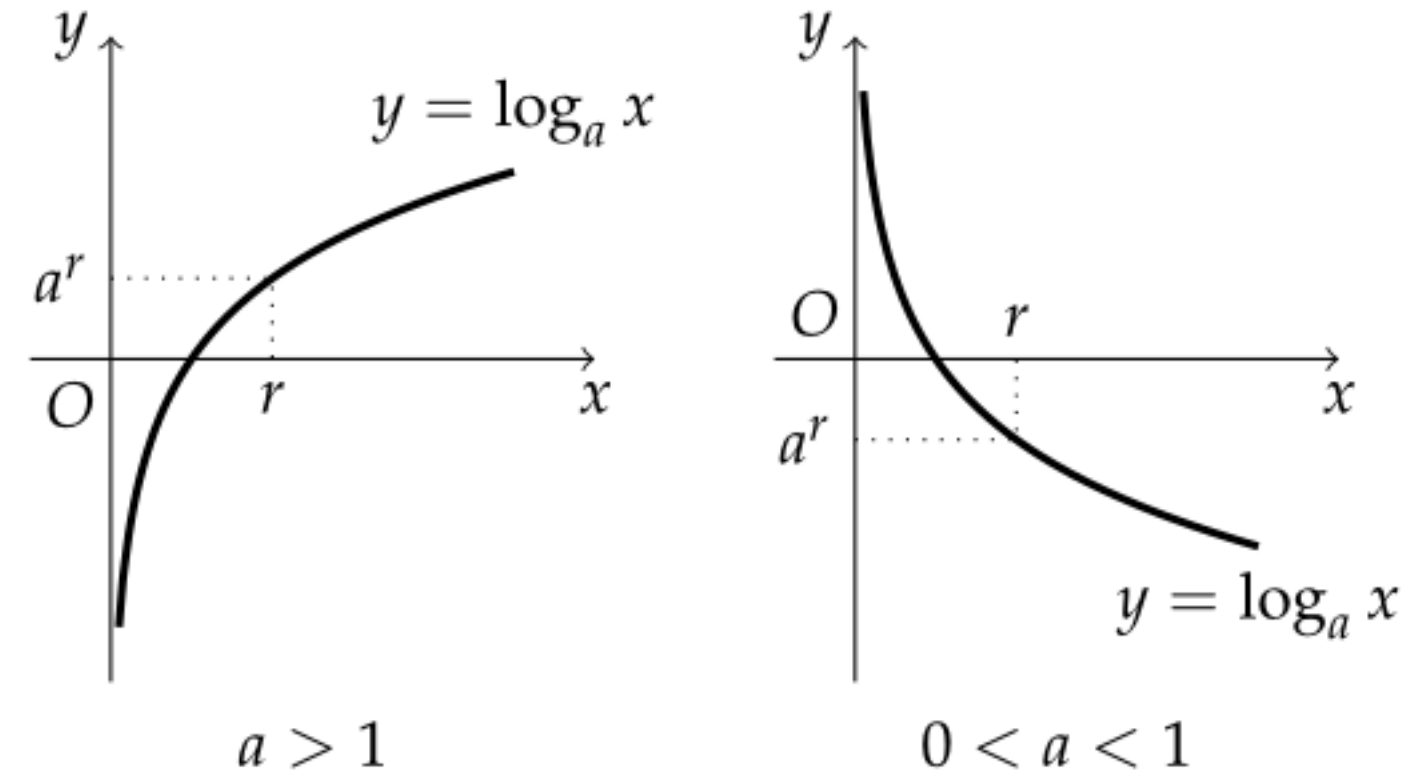
- 감염병 예측: 초기 감염자 수로 미래 확산 속도 예측
- 암세포 증식: 세포 분열 속도로 미래 종양 크기 예측

로그함수 (Logarithmic Function)

공식: $y = \log(x)$

큰 데이터를 작은단위로 다루기 좋게 변환하는 함수입니다.
처음엔 빠르게 나중에 천천히 증가하여 극값을 평평하게 만듭니다.

- 지수함수의 역함수로 지수적 성장을 평평하게 만듦
- 편차가 큰 의료 데이터를 분석 가능한 범위로 압축
- 혈중 농도: 10000, 1000, 100 $\rightarrow \log(x) \rightarrow 4, 3, 2$
- 지진 규모: 리히터 7.0이 6.0보다 10배 강함



로그는 폭발적으로 증가하거나 감소하는 데이터를 **사람이 이해할 수 있는 크기로 스케일링**해주는 함수

어디에 활용될까?

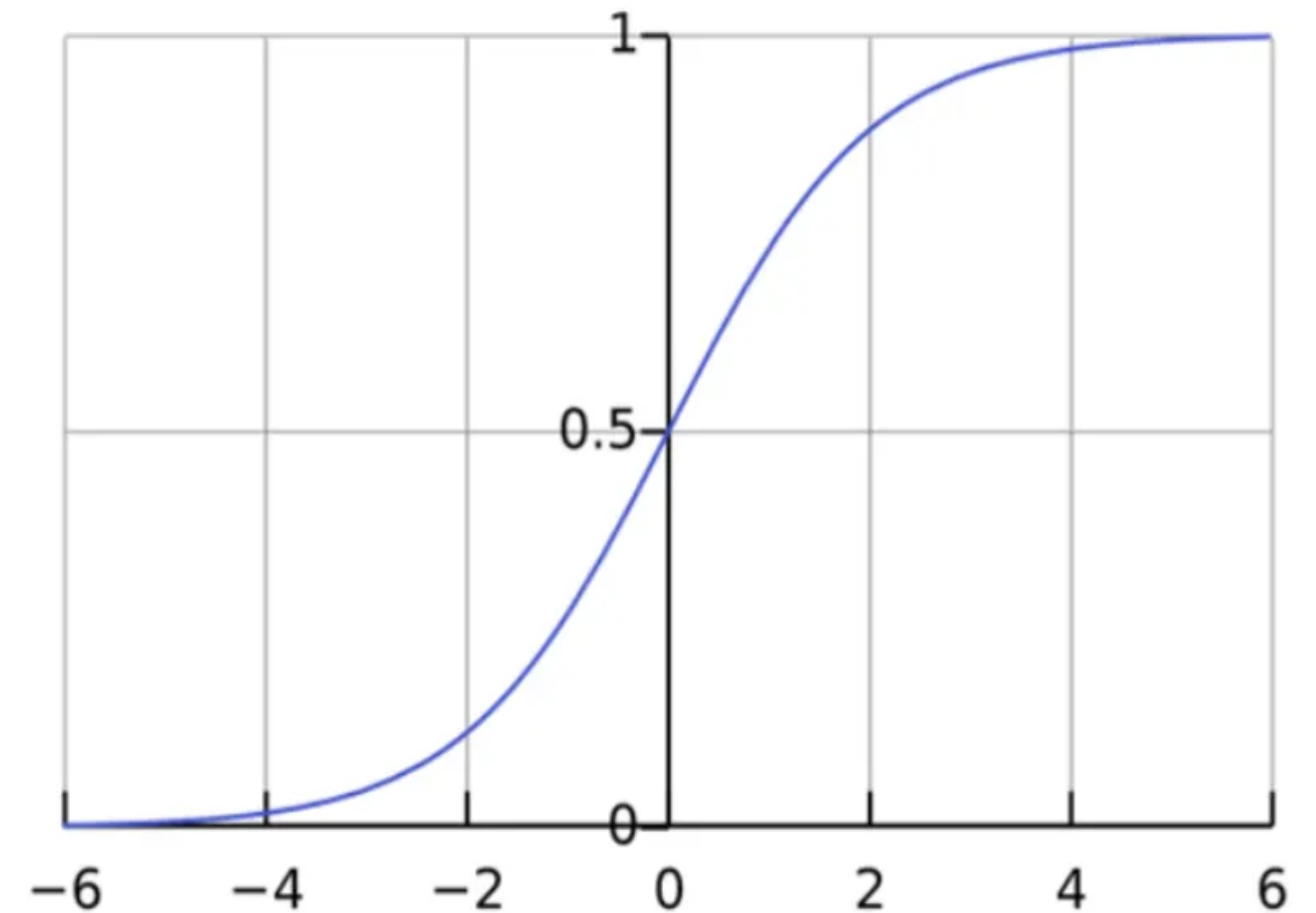
- 혈액 pH: 산성도 측정으로 정상(7.4) 범위 벗어남 감지 (확대 + 스케일링)
- 혈액검사 수치(ALT, CRP 등)의 큰 편차를 압축해 AI가 안정적으로 학습하도록 개선

시그모이드 함수 (Sigmoid Function)

공식: $y = 1 / (1 + e^x)$

모든 숫자를 0~1 사이의 확률로 바꿔주는 함수입니다.
S자 곡선 형태로 표현되며 출력값은 항상 0과 1 사이에 머무릅니다.

- 아주 작은 값은 0에, 아주 큰 값은 1에 가깝게, 애매한 값은 0.5 근처로 변환
- 중간구간에서 급격히 변화하여 AI가 민감하게 학습
- 정상/이상, 합격/불합격 같은 이진분류에 필수
- 환자 검사 결과를 0~100% 확률로 변환하여 의사결정 지원



어디에 활용될까?

- 질병 유무 분류: 증상·검사수치로 암인지 아닌지, 정상인지 이상인지 분류
- 응급 환자의 생존 확률 예측: 여러 생체신호를 0~1 확률로 변환하여 위험도를 평가

선형함수

모든 AI 예측 모델의 기본 뼈대가 되는 함수
입력이 증가하면 출력도 일정 비율로 증가/감소

지수함수

폭발적으로 급증하는 현상을 모델링하는 함수
처음엔 천천히 나중에 기하급수적 증가

로그함수

극값의 데이터를 분석 가능한 범위로 압축하는 함수
처음엔 빠르게 나중에 천천히 증가

시그모이드 함수

모든 값을 0~1 확률로 변환하여 이진분류 하는 함수
S자 곡선 모양으로 중간 구간에서 급격한 변화

미분과 경사하강법

AI가 오차를 줄이는 방향을 찾는 핵심 원리



min Cost **Weight**

머신러닝은 cost를 최소화하는 **weight**를 학습한다

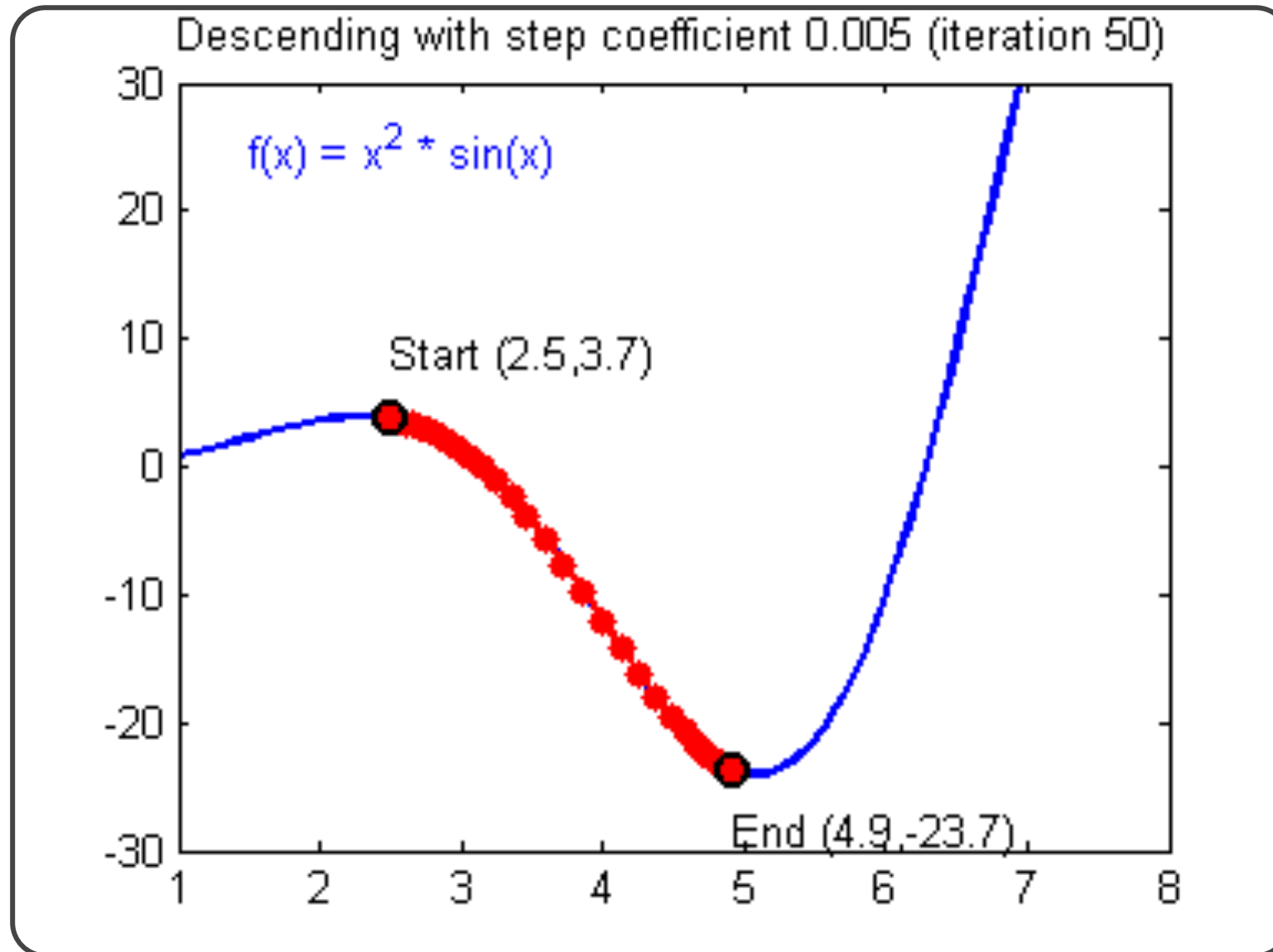


cost는 오차의 크기만 알려준다

cost는 “얼마나 틀렸는가” 만 측정하는 함수

- 하나의 예측값과 실제값의 차이 == error
- 모든 에러들을 종합적으로 계산한 결과 == cost

cost가 낮아지는 방향으로 weight를 최적화 하는 방법



방향을 알려주는 건 기울기이다

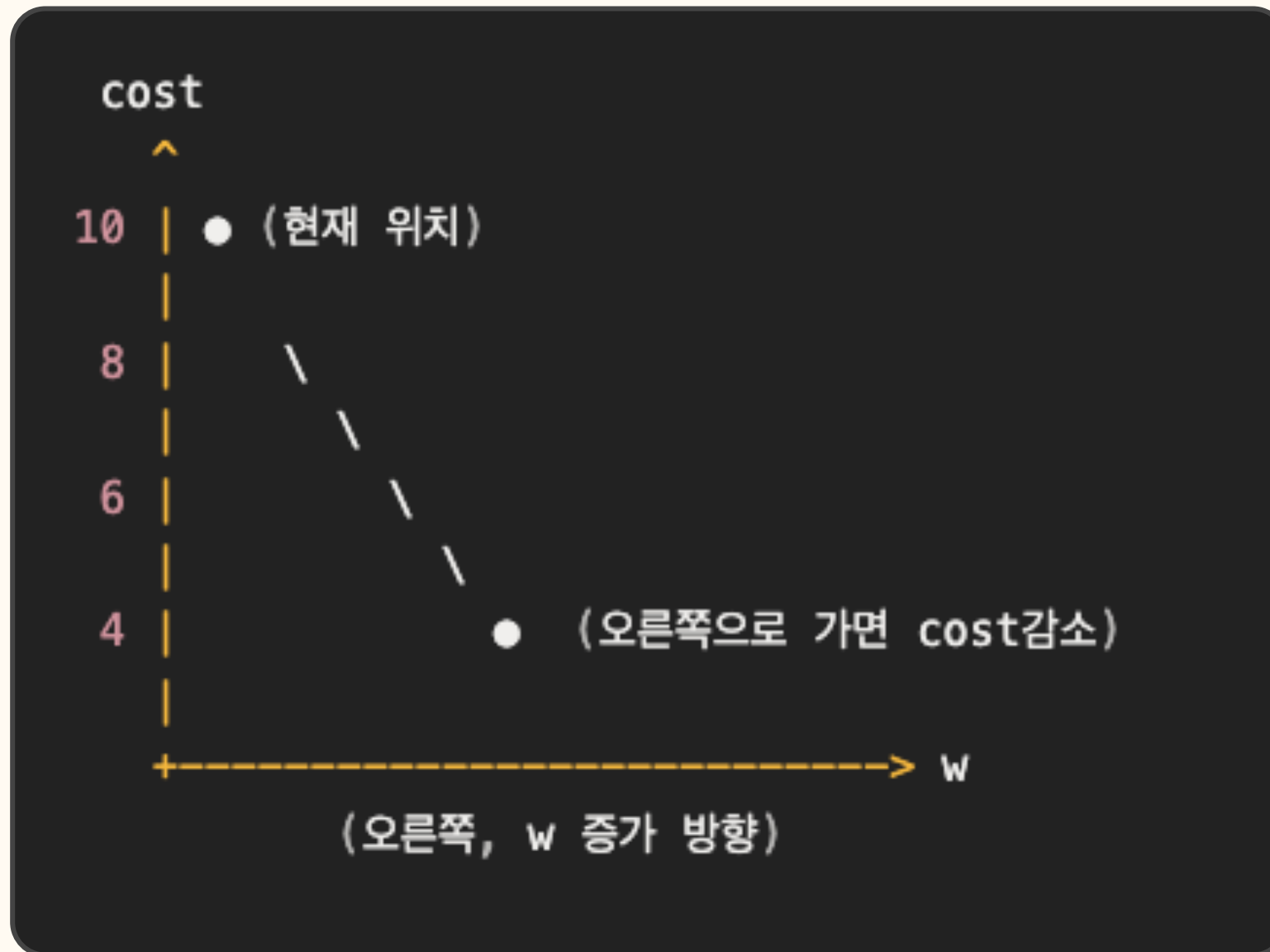
기울기(gradient)

- w 를 아주 조금 움직였을 때 cost가 어느 방향으로 변하는지 알려주는 값

$$\text{기울기(gradient)} = \frac{\Delta \text{Cost}}{\Delta \text{Weight}}$$

어떻게 계산될까?

- 기울기 $> 0 \rightarrow$ 오른쪽으로 갈수록 cost 증가 $\rightarrow$ 왼쪽으로 가야 함
- 기울기 $< 0 \rightarrow$ 오른쪽으로 갈수록 cost 감소 $\rightarrow$ 오른쪽으로 가야 함



미분: 기울기를 구하는 수학적 연산

미세하게 분해한다

미분 공식 = $\frac{\Delta y}{\Delta x} \rightarrow \frac{dy}{dx}$

x가 아주 조금 변할 때, y가 얼마나 변하는가



경사하강법이란?

오차cost가 줄어드는 **방향**으로 가중치 w 를 조금씩 **업데이트**하는 과정

경사하강법의 학습단계

- 1 가중치 (w)를 아주 조금 바꿔본다
- 2 cost가 얼마나 변하는지 확인한다
- 3 기울기가 양수 $\rightarrow w$ 를 줄이면 cost 감소
- 4 기울기가 음수 $\rightarrow w$ 를 늘리면 cost 감소
- 5 cost가 감소하는 방향으로 w 를 업데이트한다
- 6 이 과정을 반복하여 cost가 점점 내려가면 학습이 완료된다

한줄 요약

cost

오차의 크기만 알려준다. 방향은 모른다.

기울기(gradient)

지금 이 지점에서 어느 쪽이 내리막인지 방향을 알려준다.

미분

기울기를 구하는 수학 도구이다. AI는 이를이용해 스스로 더 나은 w 를 찾아간다.

경사 하강법

기울기의 반대 방향으로 w 를 조금씩 움직이며 cost를 줄이는 알고리즘이다.